

5.6 Itens: recálculo do valor da proposta e do contrato — status de entrega

Bloco 5.6 Itens: recálculo do valor da proposta e do contrato · Sprint 5 — Itens · Entregue em 2026-09-22, ajustado em 2026-09-23 · Branch: `feature-dev-breno-automacao` · Ambiente: gc-dev (`gzbmhgnpoehoꝛmnidmꝑꝑ`)

Resumo

Bloco concluído. O `valor_total` da proposta e do contrato passa a acompanhar a soma dos itens por **trigger no banco**, com regra híbrida. O formulário de edição trava obra e valor quando há itens, e a aba Itens avisa quando valor e soma divergem, com um botão para resolver. `obras_com_valores` continua batendo, provado com uma proposta aprovada.

Três migrations, aplicadas em gc-dev: `20260922120000_recalculo_valor_por_itens.sql` (o trigger), `20260922130000_recalculo_valor_trava_pai.sql` (trava contra escritas simultâneas) e `20260922140000_recalculo_valor_security_definer.sql` (tira o trigger do caminho do RLS). As duas últimas vieram do fechamento da sprint. `migration list` : **17/17 alinhadas**.

1. As quatro atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
Decidir a regra: somado dos itens ou digitado	híbrida (2.1)
Se somado: trigger no banco ou recálculo na server action	trigger (2.2)
Aviso quando <code>valor_total</code> divergir da soma dos itens	banner na aba Itens, com botão
Conferir que <code>obras_com_valores</code> continua batendo	provado por passo de escrita

2. Decisões

2.1 A regra é híbrida

- proposta/contrato **sem itens** → `valor_total` é digitado, como sempre foi;
- proposta/contrato **com itens** → `valor_total` é a soma de `itens.valor_total` .

"Sempre a soma" foi descartado com o dado na mão. Em 2026-09-22 gc-dev tinha 20 propostas **todas sem item** e com valor digitado, entre elas 3 aprovadas (PROP-2026-008/009/010) que alimentam `obras_com_valores` . Soma pura com backfill zeraria o valor dessas obras.

2.2 Trigger, e não recálculo na Server Action

Os itens não são escritos só pelo app. O workflow n8n de contrato insere `itens` direto por REST com a service key, e a rota de ingestão da automação (Fase 6) vai gravar itens de proposta. Regra no banco vale para todos; na action, só para a tela.

O trigger nasceu `security invoker` e passou a `security definer` no fechamento da sprint, porque a soma sob RLS custava o quadrado do número de itens (ver o fechamento). É seguro: ele só dispara depois que o RLS de `itens` autorizou a escrita, e só escreve no pai daquele item, que é da mesma empresa pela FK composta.

`UPDATE OF valor_unit, quantidade, proposta_id, contrato_id`: editar descrição, foto ou observação não recalcula nada.

2.3 As três escolhas que a migration registra

1. **Sem backfill.** Nenhum `valor_total` existente foi reescrito. Divergência antiga aparece no aviso e se resolve pelo botão, com a pessoa decidindo — backfill mudaria valor comercial em silêncio.
2. **Excluir o último item não zera o valor.** O pai volta a ser de valor digitado, guardando o último valor que tinha.
3. **Enquanto a soma não alcança o desconto, o valor digitado segue valendo.** Gravar a soma violaria o CHECK `desconto <= valor_total`, então o trigger não sincroniza — **e também não bloqueia**. A tela explica, e quando a soma passa do desconto o valor volta a acompanhá-la.

A escolha 3 foi **revista durante a entrega**, e vale registrar por quê. A primeira versão levantava exceção nesse caso. A camada navegador derrubou a ideia: numa proposta com desconto, o **primeiro** item lançado — ou a linha em branco de "Adicionar item" — sempre soma menos que o desconto, então era impossível começar a lançar itens. Reduzir o desconto sozinho também foi descartado: desconto é termo comercial, não é conta.

2.4 Com itens, obra e valor saem do formulário

`updateProposta` passa a:

- **recusar a troca de obra** quando há itens. `itens` tem FK composta (`proposta_id, empresa_id, obra_id`) sem `on update cascade`: trocar estouraria com erro cru, ou deixaria os itens na obra velha;
- **ignorar o `valor_total` do corpo** e gravar a soma. A tela mostra o campo travado, mas o POST é de quem mandou.

O formulário mostra as duas travas, com o motivo no hint ("Soma dos 12 itens — muda pela aba Itens", "Travada: os itens desta proposta pertencem a esta obra"). A aba Detalhes rotula o valor como "Valor total (soma de N itens)".

2.5 O aviso de divergência

Com o trigger, item nenhum cria divergência. Ela só aparece por dado anterior à migration, por escrita direta em `propostas.valor_total` (n8n, SQL) ou pela escolha 3. O banner distingue os dois casos:

- **divergência de verdade** → diz quanto sobra ou falta, e oferece "Usar a soma dos itens" (`sincronizarValorComItens`), só para quem pode editar;
- **soma abaixo do desconto** → explica que o valor digitado segue valendo até a soma alcançar o desconto, **sem** botão, porque sincronizar violaria o CHECK.

3. Um bug pego antes de qualquer tela

A primeira versão do trigger estourava em **todo** insert de item com `FOREACH expression must not be null : array_agg` de conjunto vazio devolve `NULL`, e item de proposta tem a lista de contratos vazia. Foi pego por uma prova em transação desfeita, rodada no banco antes de qualquer tela usar o trigger. Correção: `coalesce(..., '{}'::uuid[])`.

A prova final, também desfeita no fim (conferido: 0 linhas sobrando):

```
[1] 1 item 2x300      -> valor 600
[2] editar qtd 2->3    -> valor 950
[3] editar descricao  -> não dispara (UPDATE OF)
[4] soma 53 < desconto 100 -> não bloqueia, valor fica 950
[4b] soma volta a 950   -> valor 950
[5] apagar os dois itens -> valor 900 (o de 50 sincroniza em 900; apagar o último mantém)
```

4. Números reais do `npm run validar`

Rodado em 2026-09-22 contra gc-dev. Os números valem para os blocos 5.5 e 5.6 juntos.

CAMADA	ANTES (FIM DO 5.4)	AGORA	
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code>	ok
unitário	106 casos	124 casos, 0 falhas	+5 deste bloco
build	32 rotas	32 rotas, iguais à baseline	=
runtime	44/44 rotas	47/47 rotas ok	+3
dados	15/15 checagens	18/18 checagens ok	+3
escrita	73/73 passos	90/90 passos ok	+14 deste bloco
navegador	41/41 passos	41/41 passos ok , console limpo	=

Runtime (+3): SEED-DIVERGENTE-001 mostra o aviso e o botão para admin; o visualizador vê o aviso sem o botão; e o formulário de edição de SEED-ITENS-001 mostra "Soma dos 12 itens" e a obra travada. A proposta sem divergência tem asserção de que o aviso não aparece.

Dados (+3): toda proposta com itens tem valor_total igual à soma (15 registros; soma abaixo do desconto é pulada, por ser esperada); a contraprova — SEED-DIVERGENTE-001 continua divergente, senão a primeira checagem não sabe acusar; e os 3 contratos com itens batem.

Escrita (+14): o valor acompanha a soma na proposta principal do roteiro; primeiro item faz o valor virar a soma (700); excluir o último mantém 700; dois itens dão 650; baratear abaixo do desconto é aceito e o valor digitado segue valendo; quando a soma volta a alcançar o desconto o valor volta a acompanhá-la; com itens, trocar a obra é recusado; com itens, o valor_total do corpo é ignorado; escrita direta cria a divergência; sincronizarValorComItens a resolve; aprovar a proposta soma exatamente os 650 dos itens em obras_com_valores.valor_total_calculado ; e a proposta do bloco sai no fim.

A prova da view é por delta, não por valor absoluto: a obra já tem outras propostas e contratos, então o que se confere é que aprovar esta proposta acrescentou exatamente a soma dos seus itens.

4.1 Seeds

supabase/seed_itens.sql ganhou SEED-DIVERGENTE-001 : dois itens somando R\$ 5.000,00, com o valor escrito direto para R\$ 9.999,00 — é a divergência que acontece de verdade. E perdeu o update que alinhava o valor de SEED-ITENS-001 à mão: o trigger faz isso agora.

4.2 Duas correções no plano de validação

- O roteiro de navegador vazava propostas quando falhava no meio. A limpeza existia só como passo; um erro anterior pulava direto para o fim. Havia 5 RUN-★ acumuladas em gc-dev (2 de hoje, 3 de 8 e 10 de setembro), apagadas junto com os 2 itens delas. A limpeza agora roda também no finally , e só da proposta desta rodada.
- obras_com_valores não tem coluna valor_total : é valor_total_calculado . O primeiro passo da view devolveu undefined por isso — erro do teste, não do produto.

Fechamento da sprint 5 — dois bugs no trigger

Revisão de 2026-09-23: os seis documentos da sprint foram relidos, e cada pendência nominal foi corrigida ou justificada. Números do plano inteiro no fim da revisão:

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
estático	limpo	tsc --noEmit limpo, ✓ No ESLint warnings or errors
unitário	51 casos	140 casos, 0 falhas
build	31 rotas	32 rotas, iguais à baseline
runtime	41/41 rotas	47/47 rotas ok

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
dados	12/12 checagens	18/18 checagens ok
escrita	38/38 passos	101/101 passos ok
navegador	24/24 passos	50/50 passos ok, console limpo

Rodado em 2026-09-23 contra gc-dev. `migration list` : 17/17 alinhadas. Depois da rodada, conferido por query: 0 propostas de teste, 0 contratos de teste, 0 itens órfãos, 0 arquivos de item no bucket.

A revisão encontrou dois problemas no trigger, e ambos só aparecem com escrita de verdade.

1. Corrida entre escritas simultâneas. Medido: 4 inserções de item ao mesmo tempo na mesma proposta, 20 rodadas → **8 terminaram com o valor diferente da soma**. O trigger roda dentro da transação de cada insert; a transação B somava sem enxergar o item que a A acabara de inserir e ainda não confirmara, esperava o lock, e gravava por cima a sua soma velha. Acontece na importação de duas abas, na automação gravando em paralelo, e em duas pessoas lançando itens.

Correção — migration `20260922130000_recalculo_valor_trava_pai.sql` : o trigger trava a linha do pai (`for update`) **antes** de somar. B passa a esperar já na trava, e quando A confirma, a soma de B enxerga o item de A. Depois: **0 de 30 rodadas**.

2. O custo crescia com o quadrado do número de itens. Importar 500 itens pela action levava **6.654 ms**; o mesmo insert, direto no banco, **0,2 s**. O trigger era `security invoker`, e a soma que ele faz a cada item passava pela policy de `itens`, que chama `current_empresa_id()` — uma consulta a `profiles` — para cada linha somada: ~125 mil consultas para 500 itens.

Correção — migration `20260922140000_recalculo_valor_security_definer.sql` : o trigger passa a rodar como `security definer`. É seguro porque ele só dispara depois que o RLS de `itens` **já autorizou** a escrita, só escreve no pai daquele item (mesma empresa pela FK composta), e o que escreve é derivado. Depois: **427 ms** pelo PostgREST (13,6× mais rápido) e **1.087 ms** pela action. A execução direta da função foi revogada de `public`, `anon` e `authenticated`.

Nenhuma regra mudou: sem backfill, último item não zera, soma abaixo do desconto não sincroniza. A prova em transação desfeita foi rodada de novo depois das duas migrations, com o mesmo resultado.

Resolvido também:

- **O clique em "Usar a soma dos itens" (pendência 1)** — a camada navegador cria a divergência por escrita direta (como o n8n faz), vê o aviso com os dois valores, clica, e confere no banco que o valor voltou à soma.
- **Recálculo de contrato (pendência 2)** — contrato de teste com itens escritos direto no banco, como o workflow n8n de contrato faz: o valor vira a soma (250), acompanha a edição (350), e apagar todos os itens mantém o último valor.
- **Concorrência (pendência 5)** — 6 criações simultâneas numa proposta: valor == soma.

5. O que NÃO foi validado — pendências nominais

1. **Não há histórico da mudança de valor.** Quando o trigger reescreve `valor_total`, nada vai para `propostas.historico`, que é tipado para transições de status. Registrar mudança de valor muda o formato desse histórico em todas as telas — é o bloco 13.2 ("Histórico e auditoria de mudanças").
2. **Divergências antigas continuam lá, por decisão.** A migration não faz backfill; quem decide é a pessoa, pelo botão. Em gc-dev só `SEED-DIVERGENTE-001` está nessa situação, de propósito.
3. **A tela de contratos não existe** — o recálculo de contrato é provado no banco, não pela tela, que é da sprint 6.

6. Registro para quem for operar gc-prod

Três migrations deste bloco **não foram aplicadas em prod**, e vão juntas, nesta ordem, depois de `20260921150000_contatos_canal.sql`:

1. `20260922120000_recalculo_valor_por_itens.sql` — cria o trigger;
2. `20260922130000_recalculo_valor_trava_pai.sql` — trava o pai contra escritas simultâneas;
3. `20260922140000_recalculo_valor_security_definer.sql` — tira o trigger do caminho do RLS.

Aplicar só a primeira deixa prod com a corrida e com o custo quadrático. Nenhuma das três faz backfill: aplicar não muda nenhum valor existente, só passa a manter a soma dali em diante. Antes delas, o `supabase migration repair` das dez versões continua pendente em prod — ver `supabase/README.md`.

7. Próximo passo

5.7 Itens: duplicar, reordenar e ações em lote. Com o trigger no lugar, qualquer operação em lote já mantém o valor da proposta certo.